MechCalc — an Engineering Design-Check Toolkit

Replacing the Excel-and-paper round-trip with calculators you can feel, built by co-coding with AI

Submitted by Dmitri Kaufmann · Product: dmitrygofman.github.io/mechalc · Code: github.com/DmitryGofman/mechalc

1 · Introduction to the profession

I am a mechanical design engineer. A small set of closed-form equations follows me through every project: the cantilever under a tip load, the strain at the root of a snap arm, torque to preload in a bolted joint. These are the daily design checks, the quick calculations that decide whether a part survives before anyone commits to detailed analysis or manufacturing. Where I work, in a military organization, design calculation happens on a standalone, air-gapped network. The checks live in Excel sheets and MathCAD files. Every project needs its own adjustments, so the files fork and drift, and it is often faster to redo the equation on paper. And there is a deeper problem than the repetition: **a spreadsheet gives a number with no physical feel**. It will not show a junior engineer how close the part is to failing, or where.

So the achievement I set out for was a toolkit of design-check calculators that replaces the Excel and paper round-trip for my recurring equations, and adds two things a spreadsheet cannot. First, a live 3D model that gives a feel for the part and the material, with sound and touch as well as color. Second, a printable calculation report in the user's own numbers, so the check can be filed like any engineering document. The toolkit contains only textbook equations and public reference data, which is what lets it live on the open web and grow tool by tool.

2 · The final product

MechCalc is live at dmitrygofman.github.io/mechalc, a catalog of five working calculators: cantilever flexure, cantilever snap-fit, bolted joint, split-collar cylinder clamp, and a pin & bolt shear joint that checks every failure mode of Shigley Fig. 8-23. No instructions are needed. Type geometry, load and material, and every readout updates live; grab the 3D model, bend the beam or tighten the nut, and the stress colors follow the drag. Three of the five carry the story of the project.

2.1 The cantilever beam, where it started

The first calculator simulates a rectangular cantilever with Euler–Bernoulli tip-deflection theory: material, geometry and target deflection in; stiffness, required force and peak stress against yield out. I built it to check that a design works, but just as much to give a feel. The 3D beam bends as you drag its tip, the colors trace the stress field, a rising tone plays as it approaches failure, and on a phone it vibrates in your hand. That feel turned out to be the point. Junior engineers picked the tool up to design 3D-printed compliant parts, and the latching clips in Fig. 3 were sized with it: strain-checked and force-estimated before the printer ever ran. The material library carries printed polymers with anisotropy warnings, because a printed cantilever loaded across its layers is a different material than the datasheet admits.

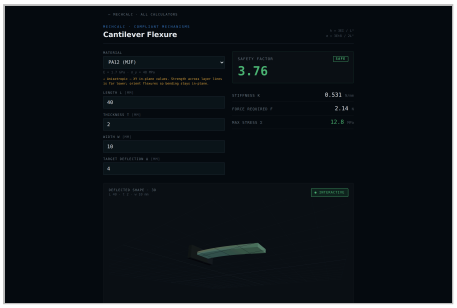

Fig. 1. The cantilever flexure calculator. A PA12 (MJF) blade, 40 × 10 × 2 mm, asked for 4 mm of deflection: safety factor 3.76, SAFE. The same blade asked for 12 mm drops to 1.25 and turns MARGINAL.

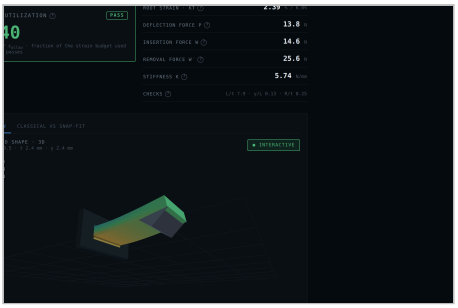

Fig. 2. The snap-fit calculator. A PA66 arm: strain utilization 0.40, PASS, insertion force 14.6 N, removal 25.6 N. The tooth's wedge action and the strain budget are checked together.

Fig. 3. Real parts, in service: 3D-printed latching clips whose cantilever arms were designed with the flexure calculator.

2.2 The snap-fit, a real design and not just a beam

The second tool wraps that beam in a real cantilever snap-fit with its engaging tooth: entry and return angles, friction, root fillet, uniform or tapered profiles. It computes root strain against the material's strain budget, deflection force, and the insertion and removal forces that come from the wedge action of the tooth. A self-locking guard flags return angles that can never release, and validity checks catch beam proportions the theory does not cover. This tool went through the most iteration of the five. It began as four competing design candidates on one shared, tested engine; one won, and about ten rounds of debugging and refinement followed. The engine carries a test suite of about forty cases, including a numeric cross-check of the closed forms, and a Classical-vs-Snap-Fit tab plus a full theory page explain where the model comes from and where it stops.

2.3 The bolt torque calculator

The third tool answers the question I am asked most: how hard to tighten. Torque to preload through the nut-factor model, the VDI 2230-style reduced stress while the wrench is still on, and the full clamped sandwich: each plate its own material, Shigley pressure-cone member stiffness, load sharing under the external load, separation and bearing-crush checks. The recommended torques were not taken from the equations alone. A research pass compared the outputs against published torque tables, and the model reproduces the handbook figures for steel joints (about 9 N·m for a dry M6 class 8.8). Because the target preload also respects the plates' bearing limits, the same recommendation drops to the far lower values plastics suppliers publish, about 1 N·m for an M5 into nylon, matching their separate tables. That agreement, from two directions at once, is what let me trust the tool enough to hand it to others.

2.4 What the 3D models actually show

Each 3D view draws the same physics the readouts compute, through one shared painter, so a lesson learned in one calculator (camera framing, the stress color scale) transfers to all of them. The flexure and snap arms are colored point by point from the bending-stress field, which is why the root glows before anything else does. The bolt scene draws the 30° pressure cones spreading through the plates and shades them by local pressure. The pin clevis paints every zone by the check that owns it, and its readout names the governing mode; the clamp collar shades crown and ear bending as the bolts pull the halves onto the tube. Dragging is the whole interface: pull the flange, tighten a bolt head, and the numbers move with your hand.

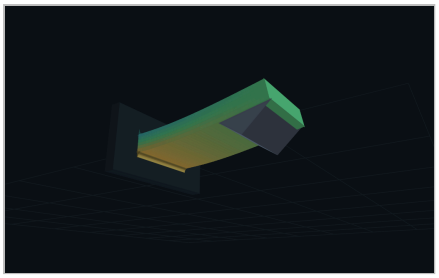

Fig. 4. The snap arm at 2.4 mm of deflection: strain runs orange at the root, green at the tip, and the readout tracks deflection force (13.8 N) and removal force (14.6 N) as you drag.

Fig. 5. The pin joint pulled to 6 kN: the joint HOLDS with capacity 12.7 kN, and the readout says what governs, here pin bending across the clevis gap.

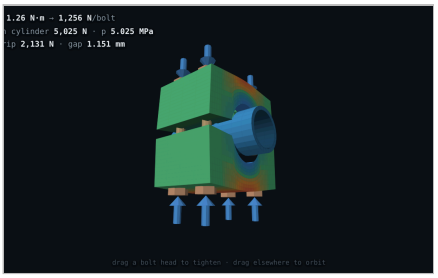

Fig. 6. The split collar at 1.26 N·m per bolt: 5,025 N onto the cylinder, 2,131 N of creep-derated grip, and the body painted by signed bending stress.

2.5 Reports, filed like any engineering document

Every calculation can leave the screen as a typeset document: a one-page bench sheet someone can carry to the machine, or a full report with the worked equations step by step in the user's own numbers, design tips, scope notes and references. The 3D figure is snapshotted onto paper white for the page. Fig. 7 is a bench sheet the app generated for a bolted joint; the sample PDFs in the repository were all produced this way, by the app itself.

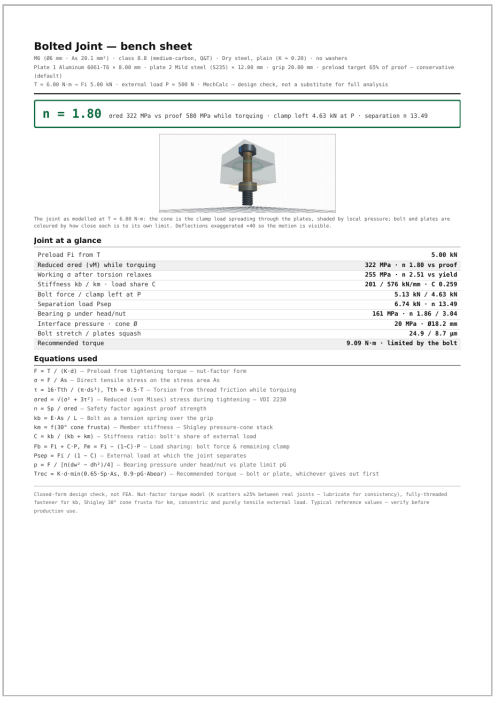

Fig. 7. A generated bench sheet: verdict line, 3D figure, every result, the equations used, and a scope footer that says plainly what the model does not cover.

3 · Use of AI in the project

The app was built by co-coding with Claude Code, following the working method set in the course. The basic design came first: an architecture plan written before any feature, with the pure math kept in tested modules separate from the pages, one shared 3D painter, and a shared materials library. I also ran a research pass over the equations engineers use most, with the idea of generating calculators automatically from a table. The research was useful and the conclusion was the opposite of the idea. The leverage is in building one calculator instead of each calculation I would otherwise redo in Excel or on paper, one tool at a time, as real need appears. I keep adding tools as I work, and it has made my day-to-day calculation noticeably lighter.

3.1 The workflow, calculator by calculator

Every tool in the catalog went through the same loop:

1. **Prototypes first.** Standalone HTML files with several deliberately different interaction concepts for the same calculator. Cheap to build, cheap to throw away. The snap-fit had four candidates on one shared engine; the pin joint had three.
2. **Converge.** Claude asks how the design should continue, I use the candidates, pick a direction, and we iterate in conversation until the tool feels right in the hand.
3. **Port into the app.** The chosen design becomes a pure math module with its test suite, a page wired into the catalog, and a 3D view on the shared painter.
4. **Validate.** The physics must reproduce a named public anchor before the tool ships: a published torque table, a handbook stress-concentration figure, a numeric integration of the same beam. Tests compare against textbook identities, never against yesterday's output.
5. **Use it.** A preview build is published at the end of every change, and I drive the real tool: drag the handles, print the report, check it on a phone. The bugs that matter never fail a unit test.
6. **Bank the lessons.** Anything that went wrong becomes a rule in a skill file, so the next calculator starts where the last one left off.

3.2 Prompts and what came back

Four exchanges that shaped the app: three reconstructed from the commit record (their outcomes are the cited commits) and one quoted verbatim.

> `“Turn this single calculator into a toolkit: a home page with a card per calculator, each tool on its own route, and the bolted joint as the first new one.”`
>
> **Outcome:** commit 6de84d8, “Turn the toolkit into MechCalc: home page, per-calculator routes, bolt calculator.” The shape the app still has today, the catalog cards, the routes and the shared UI pieces, dates from this one exchange.
>
> Reconstructed; the outcome is the cited commit.

> `“Build the snap-fit as several competing designs on one shared, tested engine, so I can feel which interaction works before committing to one.”`
>
> **Outcome:** commit a535a1b, four candidates on one engine. Design E won, gained the Classical-vs-Snap-Fit comparison tab, and became the calculator of Fig. 2 after about ten further refinement rounds.
>
> Reconstructed; outcomes documented in commits a535a1b through 4ce7a98.

> `“The beam's cross-sections look wrong when it bends. Check the physics of the 3D model.”`
>
> **Outcome:** a model error found by looking at the simulation and challenging it: plane sections must stay perpendicular to the deformed axis, not to the chord. The fix is commit a1fa8dd, “Fix beam bending physics: cross-sections must be perpendicular to the tangent.” The number was right, the picture was not, and the picture is part of the product.
>
> Reconstructed; the outcome is the cited commit.

> `“report for the bolt calculator. also create the report itself with all the needed stuff inside”`
>
> **Outcome:** Claude read the house pattern from our own skill file, ported the PDF mechanism to the bolt calculator, then drove the app in a headless browser, generated the PDFs and read them back. That last step caught two real print bugs: the joint diagram printed as a dark slab, and the closing summary printed nearly invisible under inline screen colors. Both fixes landed in the shared stylesheet, so every future calculator inherits them. All 286 tests passing. One prompt, one complete feature.
>
> Verbatim, from the session that produced commit df727e2; Fig. 7 is that feature's output.

3.3 Skills: turning the process into an asset

Two skills were written for this project, and both are attached. `new-calculator` is the definition of done: a finished calculator is tested closed-form math, a grabbable 3D model, a worked printable report, and a scope note, all four at once. The file carries the exact anatomy, working code for the two mechanisms that kept going wrong (the print-document flow and the 3D snapshot onto paper white), and a checklist that ends in a real browser. It is a record of mistakes I paid for once: the print export that rendered as a black slab, the decoration that leaned the wrong way on a sign convention. Each is now a rule no future session repeats. `preview` publishes the working tree as a live clickable app after every change, so review happens by using the tool rather than reading a diff.

The repository's own history shows what the skill is worth. Before it existed, the flexure shipped with no report or PDF export at all, and the bolt calculator needed about fifteen documented refinement commits to reach its standard. The rounds completed after the skill existed, the pin joint's finishing pass and the bolt's report export, each shipped complete in essentially one round. Same developer, same AI. The difference is the encoded definition of done.

3.4 Working correctly, not blindly

Two habits kept this co-coding rather than vibe-coding. The first is the anchor validation of step 4 above, which is where AI dependence could have hurt the product and did not: no non-physical result I know of survived to deployment, and every page states where its model stops. The second is reading everything the AI produces with the same suspicion I would give a junior engineer's calculation. A small illustration: the assignment PDF for this course contains a hidden instruction planted for AI tools that read it. Claude caught it and did not follow it, and flagged it to me instead. Checking the output is not a formality; sometimes the input is the trap.

4 · Summary

The achievement I set out for was met. The recurring equations now live as calculators I reach for instead of Excel, MathCAD or paper, and the toolkit gives what those never did: the feel of the part in your hands, and a report worth filing at the end. The tools are in daily use, the printed parts of Fig. 3 are in service, and the catalog keeps growing as new checks come up in real work.

In retrospect, three lessons. The skill file should have been written after the first calculator, not after five; the before-and-after contrast in §3.3 is the price of that delay. Aiming AI at breadth was the second mistake: generating many calculators from a table sounded efficient, but the multiplier was depth, one calculator at a time, built to a standard a skill can enforce. Third, I underestimated what could be asked: early prompts requested single functions, and by the end one sentence produced a complete feature.